

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 8 Exam Notes: Java Utility Package (Collections Framework)

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Collections Framework Overview	2
1.1	Core Interfaces	2
2	ArrayList vs. LinkedList	2
3	Accessing and Iterating Collections	3
3.1	The Iterator Interface	3
3.2	The For-Each Statement	3

1. Java Collections Framework Overview

The Collections Framework, declared inside the `java.util` package, provides a unified architecture to store and manipulate a group of objects.

Core Design Benefits

- Reduces programming effort by providing standard data structures.
- Boosts speed and quality via high-performance, optimized implementations.
- Promotes software reuse by offering polymorphic operations.

1.1 Core Interfaces

The framework is structured hierarchically around a set of core interfaces:

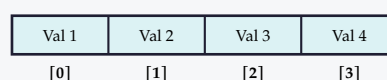
- **Collection:** The root interface defining basic operations (size, add, remove, clear).
- **List:** Extends `Collection`. Represents an ordered sequence of elements that can contain duplicates. Supports index-based access.
- **Set:** Extends `Collection`. A collection that cannot contain duplicate elements, modeling mathematical sets.
- **Queue:** Extends `Collection`. Designed to hold elements prior to processing (typically FIFO).

2. ArrayList vs. LinkedList

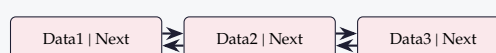
Metric	ArrayList	LinkedList
Internal Structure	Dynamically resizing array.	Doubly linked list nodes.
Access Time	$O(1)$ random access (index-based).	$O(n)$ search access (sequential traverse).
Insertion/Deletion	$O(n)$ (requires element shifts).	$O(1)$ if pointer is located.
Memory Overhead	Low (only stores elements).	Higher (stores element and prev/next links).

ArrayList vs. LinkedList Memory Layout

ArrayList (Contiguous Memory):



LinkedList (Pointer Links):



3. Accessing and Iterating Collections

Java provides two primary mechanisms to traverse through collection elements:

3.1 The Iterator Interface

An iterator provides sequential access to collection elements. It is retrieved using `iterator()` method on any Collection object.

```
ArrayList<String> list = new ArrayList<>();  
list.add("Java");  
list.add("HTML");  
  
Iterator<String> itr = list.iterator();  
while (itr.hasNext()) {  
    String element = itr.next();  
    System.out.println(element);  
}
```

- **Methods:** `hasNext()` (returns true if more elements exist), `next()` (returns next element), and `remove()` (removes current element safely during iteration).
- **Fail-Fast Behavior:** Iterators on standard collections like `ArrayList` throw a `ConcurrentModificationException` if the collection is structurally modified (add/remove) outside the iterator's own methods during iteration.

3.2 The For-Each Statement

Introduced in Java 5, the enhanced `for` loop (for-each) provides a cleaner syntax to iterate over arrays and collections:

```
for (String element : list) {  
    System.out.println(element);  
}
```

Internally, the compiler translates this syntax into standard Iterator-based code for objects implementing the `java.lang.Iterable` interface.

Quick Revision — Module 8

Java Utility package - Collections | OE-EC704A

Collection Syntax

Construct	Syntax and Example Usage
ArrayList Declaration	<code>ArrayList<String> list = new ArrayList<String>();</code>
ArrayList Insert	<code>list.add("Java"); list.add(1, "CSS"); // inserts at index 1</code>
LinkedList Declaration	<code>LinkedList<Integer> lList = new LinkedList<>();</code>
Iterator Traversal	<code>Iterator<String> itr = list.iterator(); while(itr.hasNext()) { ... }</code>
For-Each Loop	<code>for (String str : list) { System.out.println(str); }</code>

One-Line Definitions

- **Collections Framework:** Architecture of interfaces and classes for grouping multiple objects.
- **ArrayList:** Resizable-array implementation of the List interface, backing elements contiguously.
- **LinkedList:** Doubly-linked node implementation of the List and Deque interfaces.
- **Iterator:** Object traversal cursor enabling forward traversal and safe element removals.
- **Fail-Fast Iterator:** Iterator throwing exception immediately if collection structure changes during loop.
- **Iterable:** Interface indicating target object can be traversed using enhanced for-each.
- **Generics:** Compiler feature ensuring type-safety in collections (e.g., `ArrayList<String>`).
- **Auto-boxing:** Implicit compiler conversion between primitives and their wrapper classes.

Frequently Asked / Viva Questions

Q1. What is the difference between ArrayList and Vector?

`ArrayList` is non-synchronized, fast, and not thread-safe. `Vector` is synchronized, has thread-safe methods, and carries significant execution overhead.

Q2. Why does a LinkedList consume more memory than an ArrayList?

An `ArrayList` stores elements contiguously inside a backing array. A `LinkedList` wraps each element inside a node object that also holds reference addresses for the previous and next nodes.

Frequently Asked / Viva Questions (contd.)**Q3. What is fail-fast behavior in collections?**

It is a JVM runtime validation that throws a `ConcurrentModificationException` if a collection is modified structurally (elements added or deleted) while an active iterator is traversing it.

Q4. How does `ArrayList` grow its internal array capacity?

When an element is added and the array is full, `ArrayList` allocates a new array and copies elements. Starting from Java 10, it increases capacity by roughly 50% ($\text{New Capacity} = \text{Old Capacity} + (\text{Old Capacity} \gg 1)$).

Q5. What is the purpose of the `Iterable` interface?

It is the superinterface of the `Collection` interface. Implementing `Iterable` requires the class to return an `Iterator`, allowing instances to be used in the enhanced for-each loop.

Q6. Can we store primitive data types (like `int`, `char`) inside collections?

No, collections only store object references. However, Java handles this automatically using **Autoboxing**, which converts primitives to their wrapper objects (e.g., `int` to `Integer`) when inserted.

Q7. What is the difference between `Iterator` and `ListIterator`?

`Iterator` can traverse any `Collection` in a forward direction. `ListIterator` can only traverse `Lists`, but supports bidirectional traversal (forward and backward) and element modifications.

Q8. Which `List` implementation is preferred for frequent search operations?

`ArrayList` is preferred because it supports $O(1)$ constant time lookup via index-based access, whereas `LinkedList` requires $O(n)$ linear traversal.

Q9. Which `List` implementation is preferred for frequent insertions at the beginning of the list?

`LinkedList` is preferred. Inserting at the beginning requires only updating two node pointers ($O(1)$), whereas `ArrayList` requires shifting all existing elements, taking $O(n)$ time.

Q10. What is the difference between `List` and `Set` interfaces?

A `List` maintains insertion order and can contain duplicate elements. A `Set` is unordered and cannot contain duplicate elements.

ArrayList vs. LinkedList operations complexity

Operation	ArrayList	LinkedList
Get by Index	$O(1)$ constant time.	$O(n)$ linear traversal.
Insert/Delete (Start)	$O(n)$ shift overhead.	$O(1)$ pointer change.
Insert/Delete (End)	$O(1)$ amortized.	$O(1)$ tail reference update.
Memory consumption	Low (consecutive slots).	High (two references per node).